PLATFORM DOCUMENTATION · V1.0

FoodBridge

A smart food donation and poverty-support platform connecting food businesses, verified families and volunteer drivers — built for UN SDG 1 and SDG 2.

8 November 2026	React 18 + TypeScript	Supabase (PostgreSQL)	4 roles · 26 screens
Document date	Front end	Back end	Scope

Contents

1 Product overview
2 Roles and permissions
3 Public site
4 Role dashboards
5 Smart matching engine
6 Data model
7 Security and row-level security
8 Notifications and automation
9 Technical architecture
10 Internationalisation (EN/AR)
11 Configuration and deployment
12 Demo accounts
13 Operating runbook
14 Known limitations and roadmap

1. Product overview

FoodBridge is a multi-role web application that rescues surplus food from restaurants, bakeries and supermarkets and routes it to verified low-income families through volunteer drivers. It is a working application with authentication, role-based dashboards, a relational database, file storage and automated notifications — not a marketing landing page.

Problem addressed

SDG 1 — No Poverty. Food is the largest flexible expense for low-income households. Reliable food support releases money for rent, transport, school and medical costs.

SDG 2 — Zero Hunger. Roughly one third of food produced is wasted while families go without. Redirecting safe surplus closes the gap without producing anything new.

Design system

Token	Value
Primary	Forest green `#218C5A` (hover `#1B7A4E`, deep `#14603D`)
Secondary	Light green `#E8F5EC`
Background	White `#FFFFFF` and `#F6F8F7`

Text	Charcoal `#16211C`, muted `#5C6B63`
Status palette	Amber `#C77C0A` (urgent), red `#B42318` (critical/destructive)
Radius / shadow	10–16 px rounded cards, three-tier soft shadow scale
Typography	System UI stack only — no external font CDN; Arabic uses Segoe UI / Noto Sans Arabic / Geeza Pro

2. Roles and permissions

Capability	Donor	Beneficiary	Volunteer	Administrator
Create food donation	Yes	—	—	—
Edit / delete own donation	Yes	—	—	Yes (any)
Approve or reject donation	—	—	—	Yes
Browse approved donations	Own only	Yes	Assigned	All
Submit assistance request	—	Yes	—	—
Approve / reject request	—	Cancel own	—	Yes
Create match	—	—	—	Yes
Accept delivery task	—	—	Yes	Assigns
Update delivery status	—	—	Own tasks	Yes
Verify beneficiary accounts	—	—	—	Yes
View platform analytics	Own figures	Own figures	Own figures	All

> **Enforcement is three-layered.** Routes are guarded in React (`RequireAuth` / `RequireRole`), the UI hides unavailable actions, and PostgreSQL row-level security independently rejects any unauthorised query — so a tampered client cannot escalate.

3. Public site

Route	Purpose and key content
`/`	Hero with the headline “Connecting Food. Fighting Hunger. Building Hope.”, live statistics, three-step explainer, latest four donations, SDG 1 / SDG 2 section, registration call to action.
`/about`	Problem statement, the FoodBridge response, mission and vision, and how each role fits together.
`/donations`	All approved donations with free-text search plus category and location filters. Readable without an account.

`/impact`	Live metrics from database functions: meals distributed, families helped, deliveries, six-month trend chart, category bar chart, urgency donut, conversion progress bars.
`/login` · `/register`	Authentication. Registration includes a role selector; beneficiary sign-up collects household size, address and a verification note.
`*`	Localised 404 page.

4. Role dashboards

All four dashboards share one shell: a fixed 264 px sidebar (drawer below 1024 px), a sticky top bar with page title, notification bell and profile menu, and a content area on `#F6F8F7`.

4.1 Donor — `/donor`

- **Overview** — total donations, live listings, items awaiting review, meals contributed, five most recent listings.
- **My donations** — searchable table with status badges, inline edit modal and delete confirmation.
- **Add donation** — food name, category, quantity and unit, expiry date, city, pickup address, description, and a drag-and-drop photo upload (JPEG/PNG/WebP, 5 MB) written to Supabase Storage. New listings enter `pending`.
- **Incoming requests** — every match on the donor's food with the matched quantity, family size, fit score and delivery status.

4.2 Beneficiary — `/beneficiary`

- **Overview** — request counts and food received, with a verification banner while the account is `pending`.
- **Browse food** — filtered catalogue with a "Request this food" dialog that pre-fills category and a sensible quantity, validated against availability.
- **My requests** — status tracking, rejection reasons, cancellation of pending or approved requests.
- **New request** — category, quantity, household size, urgency, area and situation notes.

4.3 Volunteer — `/volunteer`

- **Overview** — availability toggle (available / busy), active tasks, completed count, open unassigned tasks, capacity and transport.
- **Delivery tasks** — accept any open task, then progress it `assigned → picked_up → in_transit → delivered`. Final step cascades the whole chain (see §8).
- **History** — completed and cancelled deliveries with timestamps.

4.4 Administrator — `/admin`

- **Overview** — registered users by role, donations, meals, deliveries, and four work queues (verifications, donation reviews, request reviews, unmatched approved requests) plus recent activity.
- **Users** — searchable, role-filtered table; verify or suspend accounts; full profile drawer including verification evidence.
- **Donations** — moderation queue by status; approve (optionally with an internal note) or reject with a reason that reaches the donor.

- **Requests** — approve, reject with a reason, review family context.
- **Smart matching** — see §5.
- **Deliveries** — assign or reassign volunteers, monitor live status, force-complete a handover.
- **Analytics** — platform KPIs, trend chart, category performance, operational health bars, CSV export.

5. Smart matching engine

Matching is deterministic application logic implemented in `src/lib/matching.ts`. No AI model, no external API. A request is compared with every approved donation and scored out of 100.

Criterion	Weight	Rule
Category fit	40	Exact category match = 40; request accepts "any food" = 20; mismatch = 0.
Quantity	20	Donation covers the request = 20; covers at least half = 12; otherwise 4.
Expiry urgency	15	Expires within 3 days = 15 (rescue priority); within 7 days = 10; later = 5; expired = 0.
Location	15	Same city = 15; different city = 5; unknown = 3.
Request urgency	10	critical 10 · high 7 · normal 4 · low 2 — this is what pushes urgent families to the top.

The UI shows the score plus a per-criterion breakdown bar, and the reasons behind each decision, so an administrator can see *why* a pairing was proposed rather than trusting a black box.

```
score = min(100, category + quantity + expiry + location + urgency)

Example — "Family Produce Mix" (produce, 40 servings, +3 days, Tampines)
matched to Fatima (produce, 15 servings, critical, Tampines):
  category  40  (exact match)
  quantity  20  (40 >= 15)
  expiry    15  (<= 3 days)
  location  15  (same city)
  urgency   10  (critical)
  ──────────────
  total     100 / 100
```

Workflow: administrator selects an approved request → reviews ranked candidates → **Create match** (status `proposed`) → **Confirm** (donation and request become `matched`) → **Assign volunteer** (creates the delivery and notifies the driver).

6. Data model

Eight tables in the `public` schema of a PostgreSQL database, all with foreign keys and constraints. Identity is owned by Supabase Auth; `public.profiles` mirrors each account.

Table	Columns	Relationships
profiles	`user_id` PK · full_name · email · phone · role · status · organization · address · city · household_size · verification_note · verified_at · created_at	Root of every relationship. Unique index on `lower(email)`.

donations	`donation_id` PK · donor_id · donor_name · food_name · category · description · quantity · unit · expiry_date · image_url · location · city · status · review_note · created_at · updated_at	`donor_id → profiles.user_id` (cascade)
assistance_requests	`request_id` PK · beneficiary_id · beneficiary_name · food_type · quantity · family_size · urgency · city · notes · status · review_note · created_at · updated_at	`beneficiary_id → profiles.user_id`
matches	`match_id` PK · donation_id · request_id · matched_quantity · score · note · status · created_at	→ donations, → assistance_requests. Unique pair `(donation_id, request_id)`.
volunteers	`volunteer_id` PK · user_id · availability · area · vehicle · capacity · active · created_at	`user_id → profiles.user_id` (unique — one profile per volunteer record)
deliveries	`delivery_id` PK · match_id · volunteer_id · pickup_address · delivery_address · pickup_date · delivery_date · status · notes · created_at · updated_at	`match_id → matches` (unique), `volunteer_id → volunteers`
notifications	`notification_id` PK · user_id · message · kind · link · is_read · created_at	`user_id → profiles.user_id`
reviews	`review_id` PK · reviewer_id · reviewed_id · rating (1–5) · comment · created_at	→ profiles twice; unique `(reviewer_id, reviewed_id)`

STATUS VOCABULARY

Entity	Allowed states
donation	`pending · approved · rejected · matched · completed · expired`
assistance request	`pending · approved · matched · fulfilled · rejected · cancelled`
match	`proposed · confirmed · completed · cancelled`
delivery	`assigned · picked_up · in_transit · delivered · cancelled`
profile	`active · pending · suspended`

REPORTING FUNCTIONS

- `platform_stats()` — totals for donations, meals, families, deliveries, each role count, open listings, active requests.
- `donation_trends()` — six months of donation and meal counts.
- `category_breakdown()` — listings and meals per food category.

- `urgency_breakdown()` — request distribution by urgency.
- `available_donations` — security-invoker view exposing approved listings without the donor identifier.

7. Security and row-level security

Every table has RLS enabled and a role-specific policy. Anonymous visitors can read only approved donations, through a column-restricted grant that excludes `donor_id`.

Table	Access rule
profiles	Own row or administrator. Insert restricted to a self-owned row with role ≠ admin.
donations	Approved rows are public; owner and administrator see everything; matched families and assigned volunteers see the listing they are involved in.
assistance_requests	Own requests, the donor of a matched donation, the assigned volunteer, or administrator.
matches	Visible to the donor, beneficiary and volunteer involved, plus administrator. Writes are administrator-only.
volunteers	Own record or administrator.
deliveries	Involved parties or administrator; unassigned tasks are visible to any active volunteer so they can be claimed.
notifications	Own rows only; no client inserts.
reviews	Reviewer or reviewed party (and administrator); one review per pair, self-reviews blocked.

> **Privilege-escalation defences.** Owners cannot change their own `role`, `status`, `verified_at` or `email`. A donor cannot approve their own donation; a beneficiary cannot approve their own request; both are silently reverted by guard triggers. Volunteers may only progress the status of their own assignment and cannot reassign it. Helper predicates (`is_admin`, `can_view_donation`, `can_view_request`, `can_view_match`, `can_view_delivery`) are `SECURITY DEFINER` with a pinned search path, which also prevents policy recursion.

Passwords are handled entirely by Supabase Auth and stored as bcrypt hashes in `auth.users`; no plaintext or password hash exists in the application schema or the client bundle.

8. Notifications and automation

Notifications are generated by database triggers, not by the browser, so they cannot be forged or missed when a request fails mid-flight.

Event	Recipient and message
Donation approved / rejected	Donor — approval confirmation, or rejection with the administrator's reason.
Request approved / rejected / fulfilled	Beneficiary — status change with any review note.

Match created	Beneficiary (“a matching donation was found”) and donor (“being scheduled for pickup”).
Delivery created	Volunteer — new assignment with the pickup address.
Delivery marked delivered	Donor, beneficiary and volunteer; also cascades match → completed, donation → completed, request → fulfilled.

The bell in the dashboard top bar polls the unread count, and `/notifications` lists everything with per-item and bulk “mark read”.

9. Technical architecture

FRAMEWORK	LANGUAGE	STYLING	ROUTING	BACKEND
React 18	**TypeScript**	**Tailwind 3**	**React Router 6**	**Supabase**

PROJECT STRUCTURE

```
src/
  App.tsx                route table + role guards
  main.tsx               entry point
  styles.css             design tokens, component classes, RTL rules
  components/
    layout/              PublicLayout, DashboardLayout, LanguageSwitch
    donations/           DonationCard, DonationFilters
    ui/                  Primitives, Form, Modal, Charts
  context/               AuthContext, ToastContext
  i18n/                  en.ts, ar.ts, LanguageContext
  lib/                   supabase, api, matching, types, format, l10n
  pages/                 public pages + donor/ beneficiary/ volunteer/ admin/
supabase/migrations/     8 versioned SQL migrations
docs/                    this document
```

Data access is centralised in `src/lib/api.ts`: one typed function per query, with errors normalised into readable messages. Components never build queries inline except for the two composite reads (donor request view, delivery claim).

Charts are hand-authored SVG components (area trend, bar, donut, progress) — no charting dependency, so the bundle stays small and the styling follows the token system exactly.

Feedback uses a toast provider for transient results, inline field errors for validation, skeleton/spinner states for loads and explicit empty states for every list.

10. Internationalisation (EN/AR)

The interface ships in English and Arabic. The choice is made in **Settings** → **Language & region** or from the switch in the site header, and is persisted per browser.

- **Full RTL mirroring** — `dir="rtl"` on the document, logical CSS properties (`ps-*`, `start-*`, `border-e`) so the sidebar, tables and cards flip without duplicate styles.
- **Localised data labels** — food categories, months, dates and relative times (for example "3 days ago" rendered in Arabic) follow the active language.
- **Shared links** — `?lang=ar` or `?lang=en` opens the app in that language.
- **Consistency rule** — when Arabic is selected, no English UI string remains; dictionary lookups fall back to English only for a missing key, never mixing text on screen.

> User-authored content (food names, notes, organisation names) is stored and displayed exactly as entered — translation applies to the platform interface only.

11. Configuration and deployment

ENVIRONMENT VARIABLES

```
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<public anon key>
```

> Only the public anon key belongs in the front end. The service-role key must never be added to this project, to a build, or to source control.

COMMANDS

```
npm install          # install dependencies
npm run dev          # local development server
npm run build        # production build into dist/
npm run preview      # preview the production build
```

BACKEND PROVISIONING

1. Create a Supabase project.
2. Apply `supabase/migrations/0001…0008` in order — schema, policies, reporting functions, triggers, storage bucket.
3. Run the seed migration to create demo accounts and sample data, or create the first administrator profile manually (see §12).
4. Set the two environment variables and rebuild.

12. Demo accounts

All seeded accounts share the password `FoodBridge123!` and are pre-confirmed. The login screen can fill any of them with one click.

Email	Role	Seeded state
`admin@foodbridge.demo`	Administrator	Full platform access
`greenleaf@foodbridge.demo`	Donor	GreenLeaf Bistro — 12 donations, 7 matched

`sunrise@foodbridge.demo`	Donor	Sunrise Bakery — bakery surplus
`freshmart@foodbridge.demo`	Donor	FreshMart — produce and pantry
`fatima@foodbridge.demo`	Beneficiary	Verified, household of 5, mixed history
`joseph@foodbridge.demo`	Beneficiary	Verified, household of 4
`mei@foodbridge.demo`	Beneficiary	**Pending** — demonstrates the verification queue
`daniel@foodbridge.demo`	Volunteer	Bedok / Tampines, motorbike, one active task
`priya@foodbridge.demo`	Volunteer	Woodlands, car, one in-transit task

> Email confirmation is enabled on the project. Seeded accounts are already confirmed; turn off "Confirm email" in Supabase → Authentication if you want new sign-ups to reach their dashboard immediately.

13. Operating runbook

ONBOARD A FOOD DONOR

1. Donor registers and selects "Food donor"; the account activates immediately.
2. They add a donation — it is created as `pending` and is not publicly visible.
3. Administrator reviews it under **Donations** and approves; the donor is notified and it appears on `/donations`.

FULFIL A FAMILY REQUEST

1. Beneficiary submits a request with urgency and household size.
2. Administrator verifies the account if still pending, then approves the request under **Requests**.
3. In **Smart matching**, pick the request, review the ranked candidates, create and confirm a match.
4. Assign a volunteer; they are notified and the task appears in their board.
5. Volunteer progresses the task to **delivered**; the donation, match and request close together and all three parties are notified.

RECOVER AN UNMATCHED DONATION

Cancel the match — the donation returns to `approved` and the request to `approved`, both available for a new pairing.

14. Known limitations and roadmap

Area	Current state	Next step
Review submission UI	Reviews table, policies and read views exist; no form to post a rating yet	Add a review prompt after a delivery is completed
Real-time updates	Views refresh on navigation and on demand	Subscribe to Supabase Realtime for the delivery board

Partial quantities	A match consumes the whole donation for its request	Decrement `donation.quantity` and allow multiple matches per donation
Matching trigger	Administrator-initiated from the matching workspace	Nightly job proposing matches automatically
Email / SMS	In-app notifications only	Transactional email through Supabase Auth hooks
Maps	Text addresses only	Optional distance scoring using geocoded coordinates

---